

Script-Controlled ACL - Restrict Record Access Based on Field Value

ServiceNow Micro-Project

PHASE 2 | REQUIREMENT ANALYSIS PHASE

2. Requirement Analysis Phase

2.1 Purpose

This phase translates the idea selected in Phase 1 into a concrete, verifiable set of functional and non-functional requirements that guide the design, development, and testing of the script-controlled ACL solution.

2.2 Stakeholders

Stakeholder	Interest / Role
System Administrator	Configures roles, tables, and ACLs; must retain full, unrestricted access to all records.
EEE Branch User (e.g., EEE User)	Should view/create/edit/delete only EEE-branch records, based on assigned roles.
Non-EEE / Unrole Users	Should be denied access to the Institution Details table entirely.
ServiceNow Platform (System Security)	Enforces the ACL rules consistently across all access channels.

2.3 Functional Requirements

ID	Requirement
FR-01	The system shall allow creation of a test user "EEE User" with a valid User ID, name, and email.
FR-02	The system shall allow creation of four custom roles — bb1, bb2, bb3, bb4 — each mapped to one CRUD operation (read, create, write, delete respectively).
FR-03	The system shall provide a custom table "Institution Details" (u_institution_details) with fields: Student Roll Number (auto-number), Student Name (reference - User), Faculty Name (reference - User), Branch (choice: ECE/EEE/CSE), Email (string), Phone Number (string), Description (multi-line string).
FR-04	The system shall contain seed records covering all three branch values (ECE, EEE, CSE) to validate filtering behavior.
FR-05	The system shall enforce a Read ACL that grants access only when the record's Branch = EEE <i>and</i> the requesting user has the bb1 role, or the user has the admin role.
FR-06	The system shall enforce a Create ACL requiring the bb2 role for creating new Institution Details records.
FR-07	The system shall enforce a Write ACL requiring the bb3 role for editing existing records.
FR-08	The system shall enforce a Delete ACL requiring the bb4 role for deleting records.
FR-09	The system shall allow verification of access behavior through the "Impersonate User" feature for both the EEE User and the Administrator.
FR-10	Users lacking the required role shall see zero records ("No records to display") when they query the table.

2.4 Non-Functional Requirements

ID	Requirement
NFR-01	Security: Access decisions must be enforced server-side by the ACL engine, not by client-side scripts, so they cannot be bypassed via direct URL/API access.
NFR-02	Consistency: The same access rule must apply uniformly across list views, forms, and the Service Portal.
NFR-03	Maintainability: ACL scripts must be short, commented, and easy to extend to additional branches or roles.
NFR-04	Auditability: Every ACL must be traceable (Updated by / Updated on) and testable via impersonation without needing separate login credentials.
NFR-05	Performance: The ACL script must execute with negligible overhead (simple role checks only, no heavy queries).
NFR-06	Usability: Administrators must retain a frictionless, full-access experience with no additional prompts.

2.5 Assumptions

- The tester has admin access to a ServiceNow developer/PDI instance and can elevate to the `security_admin` role.
- Branch-role mapping (bb1 = EEE read, etc.) is managed manually for this micro-project; it is not derived automatically from the user's profile.
- Only three branch values (ECE, EEE, CSE) are needed for the demonstration.

2.6 Constraints

- Total project timer is 3 hours, so scope is intentionally limited to one table, one branch-restriction scenario, and four CRUD ACLs.

- ACL configuration in ServiceNow requires the `security_admin` role, which must be elevated before ACLs can be created or edited.
- The "Admin overrides" checkbox and script logic must be explicit — ServiceNow does not grant admin bypass to record ACLs automatically once a script/data condition is present.

2.7 Tools & Environment

Component	Detail
Platform	ServiceNow (Application Scope: Global)
Modules used	User Administration (Users, Roles), System Definition (Tables), System Security (Access Control - ACL)
Scripting	Server-side JavaScript (GlideSystem — <code>gs.hasRole()</code>)
Verification method	User Impersonation (Administrator → EEE User → Administrator)

The requirement analysis confirms that a script-controlled ACL model with four discrete CRUD roles (bb1-bb4) and a Branch-based data condition fully satisfies the problem statement from Phase 1, within the constraints of a 3-hour micro-project.